

RV32I Pipelined Processor

Complete Project Guide

A ground-up walkthrough of every design decision,
every pattern choice, and every module.

Aayush Gelal | Adesh Pandey | Bishesh Paudel

Tribhuvan University --Electronics Minor Project 2026

Table of Contents

1. What Are We Building and Why
2. RISC-V: Why This ISA
3. The RV32I Base Integer Instruction Set
4. Architecture: Single-Cycle vs Pipeline
5. Module-by-Module Breakdown
6. The Pipeline Deep Dive
7. Pipeline Hazards: The Real Engineering Challenge
8. Performance Counters: Measuring What Matters
9. Custom ISA Extension: The ROL Instruction
10. ChaCha20 Cryptographic Benchmark
11. Test Programs Explained
12. How to Build and Run Everything
13. Project Evolution

1

What Are We Building and Why

We are building a 32-bit RISC-V processor in Verilog. Not a simulator. Not a software emulator. Actual hardware described in a hardware description language that can be synthesized onto an FPGA or fabricated as a chip.

The processor implements the RV32I base integer instruction set --the fundamental subset of RISC-V that every RISC-V core must support. On top of that, we added a 5-stage pipeline for performance, complete hazard handling so the pipeline produces correct results, performance counters to measure efficiency, and a custom instruction extension (rotate-left) to demonstrate how RISC-V's extensibility works in practice with a real cryptographic workload (ChaCha20).

This is not a toy. This processor can run real programs --it computes Fibonacci sequences, it runs ChaCha20 encryption rounds, and it handles every edge case that makes pipelined processors tricky to get right.

Project File Structure

```
RV32/
  src/
    top.v           # Top-level: 5-stage pipeline integration (401 lines)
    control.v       # Control unit / instruction decoder (140 lines)
    alu.v           # Arithmetic Logic Unit (28 lines)
    regfile.v       # 32x32-bit register file (25 lines)
    pc.v            # Program counter with enable (16 lines)
    pc_adder.v      # PC+4 adder (7 lines)
    imem.v          # Instruction memory, 64 words (15 lines)
    dmem.v          # Data memory, 64 words (19 lines)
    imm_gen.v       # Immediate value extractor (33 lines)
    program.hex     # Hazard test program
    benchmark.hex   # Fibonacci benchmark
    crypto_sw.hex   # ChaCha20 (software rotations)
    crypto_hw.hex   # ChaCha20 (hardware ROL instruction)
  tb/
    top_tb.v        # Pipeline hazard testbench
    benchmark_tb.v  # Fibonacci benchmark testbench
    crypto_tb.v     # ChaCha20 SW vs HW comparison testbench
    gen_crypto_hex.py # Python assembler for crypto benchmarks
  docs/
    RISC_CARD.pdf  # RISC-V instruction reference card
```

2

RISC-V: Why This ISA

We chose RISC-V over alternatives like ARM, MIPS, or x86 for these specific reasons:

Open standard, no licensing fees

ARM requires expensive licenses. x86 is proprietary to Intel/AMD. RISC-V is free and open --anyone can build a RISC-V processor without paying royalties. For an academic project, this means we can study the full ISA specification without legal barriers.

Clean, modular design

RISC-V was designed in 2010 at UC Berkeley with decades of hindsight from MIPS, ARM, and x86 mistakes. The base ISA (RV32I) has only ~47 instructions. Compare that to x86 which has thousands. Every instruction in RV32I has a clear purpose and a consistent encoding.

Extensibility built-in

RISC-V reserves opcode space specifically for custom extensions. This is not an afterthought --it is a core design philosophy. We used this to add our custom ROL instruction using the custom-0 opcode space (0001011). Try doing that with ARM.

Industry relevance

RISC-V is used in production by SiFive, Alibaba (T-Head), Western Digital, and others. Learning RISC-V is not an academic exercise --it is career-relevant.

3

The RV32I Base Integer Instruction Set

RV32I defines 6 instruction formats. Every instruction is exactly 32 bits wide. This fixed width simplifies instruction fetch (always read 4 bytes) and simplifies decode (fields are always in the same bit positions).

The 6 Instruction Formats

```
R-type: [funct7(7)] [rs2(5)] [rs1(5)] [funct3(3)] [rd(5)] [opcode(7)]
I-type: [imm[11:0] (12)] [rs1(5)] [funct3(3)] [rd(5)] [opcode(7)]
S-type: [imm[11:5](7)] [rs2(5)] [rs1(5)] [funct3(3)] [imm[4:0](5)] [opcode(7)]
B-type: [imm bits (7)] [rs2(5)] [rs1(5)] [funct3(3)] [imm bits(5)] [opcode(7)]
U-type: [imm[31:12] (20)] [rd(5)] [opcode(7)]
J-type: [imm bits (20)] [rd(5)] [opcode(7)]
```

R-type (Register)

Operations between two registers. ADD x3, x1, x2 means $x3 = x1 + x2$. Uses funct7 and funct3 to distinguish operations (ADD vs SUB vs AND, etc.).

I-type (Immediate)

Operations with a 12-bit constant. ADDI x2, x1, 5 means $x2 = x1 + 5$. Also used for loads: LW x6, 0(x0) loads from memory address $x0+0$.

S-type (Store)

Stores to memory. SW x8, 0(x0) stores x8 to memory at address $x0+0$. The immediate is split across two fields (bits 31:25 and 11:7) --this keeps rs1 and rs2 in the same bit positions as other formats, simplifying decode hardware.

B-type (Branch)

Conditional branches. BEQ x10, x11, +12 jumps forward 12 bytes if x10 equals x11. The immediate provides a 13-bit signed offset (the LSB is always 0 since instructions are 2-byte aligned, giving +/-4KB range).

U-type (Upper Immediate)

Loads a 20-bit immediate into the upper 20 bits. LUI x1, 0x12345 sets x1 to 0x12345000. Combined with ADDI, you can load any 32-bit constant in 2 instructions.

J-type (Jump)

Unconditional jump. JAL x1, offset jumps to PC+offset and saves the return address (PC+4) in x1. The 21-bit offset gives +/-1MB range.

Why Are immediates Scrambled?

This is one of the cleverest parts of RISC-V. The sign bit is always bit[31] in every format, so sign-extension hardware is trivial (just replicate bit[31]). Also, rs1 is always bits[19:15] and rs2 is always bits[24:20] regardless of format. The immediate bits were deliberately shuffled so that register fields never move. This saves mux levels in the decode stage.

Instructions We Implement

Category	Instructions	Count
Arithmetic	ADD, SUB, ADDI	3
Logical	AND, OR, XOR, ANDI, ORI, XORI	6
Shifts	SLL, SRL, SRA, SLLI, SRLI, SRAI	6
Compare	SLT, SLTU, SLTI, SLTIU	4
Load/Store	LW, SW	2
Branch	BEQ, BNE, BLT, BGE, BLTU, BGEU	6
Jump	JAL, JALR	2
Upper Imm	LUI, AUIPC	2
Custom	ROL (rotate left)	1
Total		32

4

Architecture: Single-Cycle vs Pipeline

The Single-Cycle Starting Point

Our project started as a single-cycle processor where every instruction completes in one clock cycle:

```
PC -> IMEM -> Decode -> RegFile Read -> ALU -> DMEM -> RegFile Write
all in one clock cycle
```

Advantage: Simple. CPI (Cycles Per Instruction) is exactly 1.00.

Problem: The clock period must be long enough for the slowest instruction (a load: IMEM read + register read + ALU + DMEM read + register write). The clock frequency is limited by this critical path.

Why We Moved to a 5-Stage Pipeline

A pipeline breaks each instruction into 5 stages. Each stage takes one shorter clock cycle. Multiple instructions overlap in execution:

Cycle:	1	2	3	4	5	6	7
Instr 1:	IF	ID	EX	MEM	WB		
Instr 2:		IF	ID	EX	MEM	WB	
Instr 3:			IF	ID	EX	MEM	WB

Clock frequency boost: Each stage does $\sim 1/5$ of the work, so the clock can run $\sim 5\times$ faster. An instruction still takes 5 cycles to complete (latency), but a new instruction finishes every cycle (throughput).

The trade-off: Pipelining introduces hazards --situations where the next instruction depends on a result that is not yet available. Our pipeline achieves CPI of ~ 1.14 on Fibonacci, meaning only 14% overhead from hazards, while gaining $\sim 5\times$ clock frequency. Net throughput improvement: $\sim 4.4\times$ over single-cycle.

Harvard Architecture

We use a Harvard architecture --separate instruction memory (IMEM) and data memory (DMEM). In a pipeline, the IF stage reads from instruction memory while the MEM stage simultaneously reads/writes data memory. With a single memory (Von Neumann), these would conflict. Two separate memories eliminate structural hazards.

5

Module-by-Module Breakdown

5.1 Program Counter (pc.v)

```
module pc(
    input clk, input rst, input en,
    input [31:0] pc_next,
    output reg [31:0] pc
);
    always @(posedge clk) begin
        if (rst)      pc <= 32'b0;
        else if (en)  pc <= pc_next;
    end
endmodule
```

The PC is a 32-bit register holding the address of the instruction being fetched. On every rising clock edge, it updates to pc_next --unless reset is active (PC goes to 0) or enable is low (PC holds its value). The enable signal implements stalling: when a load-use hazard is detected, en goes low, and the PC freezes for one cycle.

Design choice: A dedicated enable signal is cleaner than muxing pc_next to feed back the current PC. It clearly separates 'what is the next PC' from 'should we update at all' and maps directly to clock-enable on a real flip-flop.

5.2 PC Adder (pc_adder.v)

```
module pc_adder(
    input [31:0] a,
    output [31:0] y
);
    assign y = a + 32'd4;
endmodule
```

Adds 4 to the current PC because each instruction is 4 bytes wide. PC+4 is needed in three places: as the next sequential PC, as the return address for JAL/JALR, and it propagates through the pipeline to the WB stage.

5.3 Instruction Memory (imem.v)

```
module imem(
    input [31:0] a,
    output [31:0] rd
);
    reg [31:0] RAM [63:0]; // 64 words = 256 bytes
    integer i;
    initial begin
        for (i = 0; i < 64; i = i + 1)
```



```

        RAM[i] = 32'h00000013; // Fill with NOPs
        $readmemh("src/program.hex", RAM);
    end
    assign rd = RAM[a[31:2]]; // Word-addressed
endmodule

```

64 words (256 bytes). Word-addressed via a[31:2] --the PC counts in bytes (0, 4, 8...) but memory is organized as 32-bit words. Dropping the low 2 bits converts byte address to word index (implicit divide by 4).

NOP initialization (0x00000013): Every location is filled with NOP (ADDI x0, x0, 0) before loading the program. This prevents undefined 'x' values from propagating through the pipeline if the PC overshoots the program. The hex value decodes as 'add 0 to x0 and store in x0' --a true no-operation.

5.4 Immediate Generator (imm_gen.v)

```

always @(*) begin
    case (instr[6:0])
        7'b0010011, 7'b0000011: // I-type
            imm_ext = {{20{instr[31]}}, instr[31:20]};
        7'b0100011: // S-type
            imm_ext = {{20{instr[31]}}, instr[31:25], instr[11:7]};
        7'b1100011: // B-type
            imm_ext = {{20{instr[31]}}, instr[7],
                instr[30:25], instr[11:8], 1'b0};
        7'b1101111: // J-type (JAL)
            imm_ext = {{12{instr[31]}}, instr[19:12],
                instr[20], instr[30:21], 1'b0};
        7'b0110111, 7'b0010111: // U-type (LUI, AUIPC)
            imm_ext = {instr[31:12], 12'b0};
    endcase
end

```

Extracts the immediate from different formats and sign-extends to 32 bits. The expression {20{instr[31]}} replicates the sign bit 20 times. If bit 31 is 1 (negative), we get 0xFFFFF...; if 0 (positive), we get 0x00000...

B-type and J-type append 1'b0 at the LSB --branch/jump offsets are always multiples of 2 (instructions are at least 2-byte aligned). This implicit zero gives an extra bit of range without costing a bit in the instruction encoding.

U-type appends 12'b0 --LUI/AUIPC place the immediate in the upper 20 bits. Combined with ADDI (lower 12 bits), any 32-bit constant can be constructed in 2 instructions.

5.5 Control Unit (control.v)

Purely combinational decoder: takes opcode, funct3, and funct7 from the instruction and produces all control signals for the datapath.

Signal	Width	Purpose
reg_write	1	Enable register file write in WB
alu_src	1	ALU src B: 0=register, 1=immediate
alu_control	4	Which ALU operation to perform

mem_write	1	Enable data memory write in MEM
result_src	2	WB source: 00=ALU, 01=mem, 10=PC+4
branch	1	Conditional branch instruction
jump	1	JAL instruction
jalr	1	JALR instruction
alu_src1	2	ALU src A: 00=reg, 01=PC, 10=zero

Why separate jump and jalr? Both are unconditional jumps, but they compute the target differently: JAL uses PC + immediate (PC-relative), JALR uses register + immediate (register-indirect) with LSB cleared. They need different mux selections.

Why alu_src1 has 3 options? Most instructions use a register for ALU source A. But AUIPC needs PC + immediate (source A = PC), and LUI needs 0 + immediate (source A = 0). Rather than adding separate adders, we reuse the ALU by muxing its input. Area-efficient.

The R-type funct7 trick: ADD and SUB share the same opcode and funct3. They differ only in funct7 bit 5 (ADD=0, SUB=1). Same for SRL/SRA. This RISC-V encoding decision minimizes decode complexity --one bit distinguishes the operation.

5.6 Register File (regfile.v)

```
module regfile(
    input clk, input we3,
    input [4:0] a1, a2, a3,
    input [31:0] wd3,
    output [31:0] rd1, rd2
);
    reg [31:0] rf [31:0];
    always @(posedge clk) begin
        if (we3) rf[a3] = wd3;
    end
    assign rd1 = (a1 == 5'b0) ? 32'b0 :
        (we3 && a3 == a1 && a3 != 5'b0) ? wd3 : rf[a1];
    assign rd2 = (a2 == 5'b0) ? 32'b0 :
        (we3 && a3 == a2 && a3 != 5'b0) ? wd3 : rf[a2];
endmodule
```

32 registers x 32 bits. x0 is hardwired to zero (first check). Dual-read, single-write. Synchronous write on clock edge, asynchronous (combinational) read.

The write-through bypass (second condition in each assign) is critical: when WB writes a register in the same cycle that ID reads it, the bypass short-circuits the write data to the read output. Without this, a 3-instruction gap between producer and consumer would read stale data. The forwarding unit only covers 1 and 2-instruction gaps. This bypass was a real bug we discovered and fixed (see Section 7.4).

5.7 ALU (alu.v)

```
always @(*) begin
    case (alu_control)
```

```

4'b0000: result = src1 + src2;          // ADD
4'b0001: result = src1 - src2;          // SUB
4'b0010: result = src1 & src2;          // AND
4'b0011: result = src1 | src2;          // OR
4'b0100: result = src1 ^ src2;          // XOR
4'b0101: result = ($signed(src1) < $signed(src2)) ? 1 : 0; // SLT
4'b0110: result = src1 << src2[4:0];    // SLL
4'b0111: result = src1 >> src2[4:0];    // SRL
4'b1000: result = (src1 < src2) ? 1 : 0; // SLTU
4'b1001: result = $signed(src1) >>> src2[4:0]; // SRA
4'b1010: result = (src1 << src2[4:0])
                | (src1 >> (32 - src2[4:0])); // ROL
    endcase
end
assign zero = (result == 0);

```

Purely combinational. 11 operations. Shift amounts use only src2[4:0] (5 bits for 0-31 range). \$signed() enables two's complement comparison for SLT. SRA uses >>> (arithmetic shift) to preserve the sign bit. ROL combines left-shift and right-shift to achieve rotation in one operation.

The zero flag is asserted when result == 0, used by branch logic: BEQ subtracts operands --zero means equal. BNE checks ~zero.

5.8 Data Memory (dmem.v)

```

module dmem(
    input clk, input we,
    input [31:0] a, wd,
    output [31:0] rd
);
    reg [31:0] RAM [63:0];
    assign rd = RAM[a[31:2]]; // Async read
    always @(posedge clk)
        if (we) RAM[a[31:2]] <= wd; // Sync write
endmodule

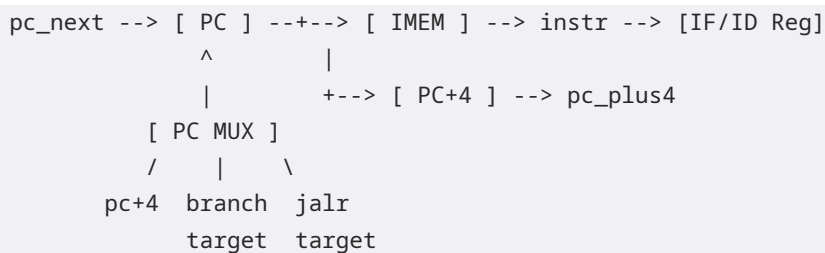
```

64 words (256 bytes). Asynchronous read (combinational, same cycle), synchronous write (on clock edge). Same a[31:2] word-addressing as IMEM. No initialization --programs must write before reading.

6

The Pipeline Deep Dive

6.1 Stage 1: Instruction Fetch (IF)



The PC outputs the current address. IMEM reads the instruction (combinational). The PC adder computes PC+4. The PC-next mux selects: JALR target (highest priority), JAL/branch target, or PC+4 (default sequential). Everything latches into IF/ID on the clock edge.

The IF/ID register has three behaviors: (1) Reset or flush: insert NOP (0x00000013) to cancel the speculatively fetched instruction. (2) Stall: hold current values so the instruction stays for an extra cycle. (3) Normal: latch new instruction, PC, PC+4.

6.2 Stage 2: Instruction Decode (ID)

The control unit decodes if_id_instr into control signals. The register file reads rs1 and rs2 (bits [19:15] and [24:20]). The immediate generator extracts and sign-extends the immediate. Everything latches into ID/EX.

On stall or flush, ID/EX gets a bubble --all control signals zeroed. A bubble is an instruction that will not write registers, not write memory, not branch. It flows through the pipeline doing nothing.

6.3 Stage 3: Execute (EX)

The most complex stage. Contains forwarding muxes, ALU source muxes, the ALU itself, branch resolution, and target computation.

Forwarding muxes select between: the register file value (no hazard), the EX/MEM value (1-instruction gap), or the MEM/WB value (2-instruction gap). The ALU source A mux further selects between the forwarded register value, PC (AUIPC), or zero (LUI). The ALU source B mux selects between the forwarded register value or the immediate.

Branch resolution uses the ALU result: BEQ/BNE check the zero flag (SUB result). BLT/BGE check bit[0] of SLT result. BLTU/BGEU check bit[0] of SLTU result. This reuses the ALU for comparison instead of adding a separate comparator.

JALR target = ALU result & 0xFFFFFFF. The ALU computes rs1 + immediate, then the LSB is masked to zero per the RISC-V spec (ensures 2-byte alignment).

6.4 Stage 4: Memory Access (MEM)

The simplest stage. For loads, DMEM reads from the address computed by the ALU in EX. For stores, DMEM writes the forwarded rs2 value to the computed address. For all other instructions, nothing happens --the ALU result just passes through.

Important: the store data (ex_mem_rd2) uses the forwarded value from the EX stage, not the raw register value. This handles the case where a store needs a value that was just computed by the previous instruction.

6.5 Stage 5: Write Back (WB)

A 3-way mux selects what to write back to the register file: ALU result (for arithmetic/logic), memory data (for loads), or PC+4 (for JAL/JALR return address). The write happens through the register file's write port.

7

Pipeline Hazards: The Real Engineering Challenge

Hazards are situations where the next instruction cannot execute correctly because it depends on information that is not yet available. Without hazard handling, our pipeline would produce wrong results. This is where most of the engineering complexity lives.

7.1 Data Hazards and Forwarding

The Problem

Consider these back-to-back instructions:

```
ADDI x1, x0, 5    # x1 = 5      (result in EX, not yet in regfile)
ADDI x2, x1, 1    # x2 = x1 + 1 (reads x1 from regfile --STALE!)
```

When the second instruction reads x1 in ID, the first has not yet written its result to the register file (that happens in WB, 2 stages later). This is a Read-After-Write (RAW) hazard.

The Solution: Forwarding

The result already exists in the EX/MEM or MEM/WB pipeline register. We grab it from there instead of from the register file.

```
// Forwarding unit (for rs1):
if (ex_mem_reg_write && ex_mem_rd != 0
    && ex_mem_rd == id_ex_rs1)
    forward_a = 2'b10; // From EX/MEM (1-instr gap)
else if (mem_wb_reg_write && mem_wb_rd != 0
        && mem_wb_rd == id_ex_rs1)
    forward_a = 2'b01; // From MEM/WB (2-instr gap)
else
    forward_a = 2'b00; // No forwarding needed
```

Three conditions must be true: the earlier instruction writes a register, the destination is not x0, and the destination matches the source being read. EX/MEM has priority over MEM/WB (more recent result wins).

7.2 Load-Use Hazard and Stalling

The Problem

```
LW    x6, 0(x0)    # x6 = mem[0] (data available END of MEM)
ADDI  x7, x6, 1    # x7 = x6 + 1 (needs x6 START of EX)
```

The load data does not exist anywhere in the pipeline yet --there is nothing to forward. We must stall for one cycle.

The Solution: 1-Cycle Stall

```
assign stall = id_ex_result_src == 2'b01 && // Load in EX
              id_ex_rd != 5'b0 &&           // Not x0
              (id_ex_rd == if_id_instr[19:15] || // rs1
               id_ex_rd == if_id_instr[24:20]); // rs2
```

When stall asserts: PC freezes (same instruction in IF), IF/ID holds (same instruction in ID), ID/EX gets a bubble. After one cycle, the load data is in MEM/WB and normal forwarding delivers it.

7.3 Control Hazards and Flushing

The Problem

```
BEQ x10, x11, target # Resolved in EX stage
ADDI x12, x0, 1      # Already in ID -- WRONG if branch taken!
ADDI x13, x0, 2      # Already in IF -- WRONG if branch taken!
```

We do not know if the branch is taken until EX resolves it. By then, wrong instructions have entered the pipeline.

The Solution: Flush

```
assign flush = ex_branch_taken || id_ex_jump || id_ex_jalr;
```

When flush asserts, IF/ID gets a NOP (cancels instruction in ID) and ID/EX gets a bubble (cancels instruction that was in ID). Each taken branch or jump wastes 1 cycle. We chose not to implement branch prediction --the penalty is only ~10% of cycles, and the basic flush mechanism is more educational.

7.4 The Write-Through Bypass Bug

This was a real bug discovered during Fibonacci testing. Scenario: 3-instruction gap between producer and consumer:

```
ADDI x1, x0, 5      # Writes x1 in WB (cycle N)
NOP                 # gap 1
NOP                 # gap 2
ADDI x2, x1, 1      # Reads x1 in ID (cycle N) -- SAME CYCLE!
```

The register file read is combinational (before the clock edge) but the write is synchronous (on the clock edge). Without the bypass, the read gets the OLD value. The forwarding unit does not help --it only checks EX/MEM and MEM/WB, but the writing instruction is in WB (past MEM/WB). The fix is the write-through bypass in the register file: if WB writes the same register ID reads, bypass the write data directly to the read output.

8

Performance Counters: Measuring What Matters

We built hardware performance counters directly into the pipeline to quantitatively analyze how well it performs.

Counter	What It Measures
perf_cycles	Total clock cycles elapsed
perf_instrs	Instructions retired (completed WB with valid=1)
perf_stalls	Cycles wasted on load-use stalls
perf_flushes	Cycles wasted on branch/jump flushes
perf_halted	Freezes all counters when JAL x0,0 detected

CPI formula: $\text{cycles} / \text{instructions} = 1.0 + (\text{stalls} + \text{flushes}) / \text{instructions}$

Halt detection: Programs end with JAL x0, 0 (jump to self, write to x0 which is discarded). We detect this pattern and freeze counters. Without it, the processor would endlessly execute the halt loop, making metrics meaningless.

Fibonacci Benchmark Results

Metric	Single-Cycle	5-Stage Pipeline
CPI	1.00	1.14
Stall cycles	0	0 (forwarding resolves all)
Flush cycles	0	8 (taken branches)
Max clock freq	$\sim 1/T_{\text{crit}}$	$\sim 5\times$ higher
Effective throughput	$1/T_{\text{crit}}$	$\sim 4.4\times$ better

Zero stalls: The Fibonacci code has no load-use hazards --all values flow through registers and forwarding handles everything. The 8 flushes come from 7 taken loop branches + 1 halt jump.

Without forwarding, CPI would be $\sim 1.6\text{-}2.0\times$ on this workload (every data dependency would require stalling), proving the forwarding unit's hardware cost is justified.

9

Custom ISA Extension: The ROL Instruction

Why Extend the ISA

RISC-V is designed to be extended. The base ISA is minimal --no multiply, divide, floating point, or bitwise rotations. These are left to optional or custom extensions. We added Rotate Left (ROL) to demonstrate extensibility with a real workload.

Instruction Encoding

```

Bit layout: 0000000 | rs2 | rs1 | 001 | rd | 0001011
             funct7  rs2   rs1  funct3  rd   opcode

Opcode:      0001011 (custom-0 reserved space)
Format:      R-type
Assembly:    ROL rd, rs1, rs2
Operation:   rd = (rs1 << rs2[4:0]) | (rs1 >> (32 - rs2[4:0]))

```

What Changed in Hardware

Only 3 changes were needed --demonstrating clean modular design:

- ALU (alu.v): Added one case --4'b1010: result = (src1 << src2[4:0]) | (src1 >> (32 - src2[4:0]))
- Control (control.v): Added one opcode case --7'b0001011: reg_write=1, alu_src=0, alu_control=4'b1010
- Pipeline, forwarding, stall logic: NO changes. ROL flows through the pipeline exactly like ADD. The pipeline does not need to know what the ALU computes.

10

ChaCha20 Cryptographic Benchmark

10.1 What is ChaCha20

ChaCha20 is a stream cipher designed by Daniel J. Bernstein, used in TLS 1.3 (HTTPS), WireGuard VPN, Google's QUIC protocol, and SSH. It was designed to be fast in software on processors lacking AES hardware. Its core operation is the quarter-round, using only: addition, XOR, and bitwise rotation.

No multiplication, no table lookups, no variable-time operations. This makes it resistant to side-channel attacks and efficient on simple processors --like ours.

10.2 The Quarter-Round Function

```
a += b; d ^= a; d <<= 16;
c += d; b ^= c; b <<= 12;
a += b; d ^= a; d <<= 8;
c += d; b ^= c; b <<= 7;
```

Four lines, each with ADD + XOR + ROTATE. The rotation amounts (16, 12, 8, 7) are constants chosen by Bernstein for optimal diffusion. Our benchmark runs 4 rounds on initial values a=0x61, b=0x62, c=0x63, d=0x64.

Expected output (verified by Python golden model):

```
a = 0xD576E19B
b = 0x1898E8F8
c = 0x36858953
d = 0x9FF722DF
```

10.3 Software vs Hardware Rotation

Software (standard RV32I): 3 instructions per rotation

```
SLLI x5, x4, 16    # t0 = d << 16
SRLI x6, x4, 16    # t1 = d >> 16
OR   x4, x5, x6    # d = t0 | t1 (rotated)
```

Three instructions, two temporary registers (x5, x6). Each quarter-round has 4 rotations = 12 instructions just for rotations out of 22 total.

Hardware (custom ROL): 1 instruction per rotation

```
ROL  x4, x4, x12    # d = ROL(d, 16) -- one cycle!
```

One instruction, no temporaries. Loop body shrinks from 22 to 14 instructions.

10.4 Results and Why Our Solution Was Great

Metric	Software (RV32I)	Hardware (ROL)	Improvement
Instrs per iteration	22	14	36% fewer
Temporary registers	2 (x5, x6)	0	Frees registers
Cycles per rotation	3-4	1	3-4x faster

Why this solution is great:

- Minimal hardware cost: ONE line in the ALU, ONE case in the control unit. The gate count increase is tiny --barrel shifter logic was already mostly present for SLL/SRL.
- Measurable real-world impact: ChaCha20 is used in production TLS. The 36% instruction reduction translates to lower energy and higher throughput.
- Standard extension path: Our ROL matches the RISC-V Zbb (Bitmanip) extension's semantics. We used custom-0 opcode space, but the concept is the same one the official extension standardizes.
- Clean verification: Python golden model computes exact expected output. Both SW and HW versions produce identical results, verified automatically by the testbench.
- Full-stack demonstration: Algorithm -> Python assembler -> machine code hex -> hardware execution -> automated verification. Shows complete hardware/software co-design.

11

Test Programs Explained

program.hex --Pipeline Hazard Tests

Systematically tests every hazard scenario:

Test 1: EX/MEM Forwarding (back-to-back)

```
ADDI x1, x0, 5
ADDI x2, x1, 1    -> forward x1 from EX/MEM
```

Test 2: MEM/WB Forwarding (2-instruction gap)

```
ADDI x4, x0, 10
NOP                      -> 1 gap
ADDI x5, x4, 4    -> forward x4 from MEM/WB
```

Test 3: Load-Use Hazard (stall + forward)

```
LW  x6, 0(x0)    -> load from memory
ADDI x7, x6, 1    -> stall 1 cycle, then forward
```

Test 4: R-type Forwarding Chain

```
ADD  x1, x1, x2    -> forward both inputs
ADD  x2, x1, x1    -> forward x1 from EX/MEM
```

Test 5: Store with Forwarded Data

```
ADDI x8, x0, 20
SW  x8, 0(x0)    -> store with forwarded x8
```

Test 6: Branch with Forwarding

```
ADDI x10, x0, 5
ADDI x11, x0, 5
BEQ  x10, x11, +12 -> forward both, branch taken
ADDI x12, x0, 1    -> FLUSHED
ADDI x12, x0, 0    -> branch target
```

benchmark.hex --Fibonacci

Init: x1=0 (F0), x2=1 (F1), counter=2, limit=10

Store: mem[0]=0, mem[4]=1

Loop:

```
x3 = x1 + x2      # F(n) = F(n-2) + F(n-1)
mem[ptr] = x3      # store F(n)
x1 = x2           # shift
x2 = x3           # shift
ptr += 4, counter++
```

```
BNE counter, 10, loop
```

```
Result: mem[0..9] = 0, 1, 1, 2, 3, 5, 8, 13, 21, 34
```

12

How to Build and Run Everything

Prerequisites: Icarus Verilog (iverilog), GTKWave (optional), Python 3 (optional).

```
# Pipeline hazard test
iverilog -o hazard_test src/*.v tb/top_tb.v && vvp hazard_test

# Fibonacci benchmark
iverilog -o bench src/*.v tb/benchmark_tb.v && vvp bench

# ChaCha20 crypto benchmark (SW vs HW)
iverilog -o crypto src/*.v tb/crypto_tb.v && vvp crypto

# Regenerate crypto hex files
python3 gen_crypto_hex.py

# View waveforms
gtkwave cpu_test.vcd
```

13

Project Evolution

Our git history tells the story of how this project grew:

Phase 1: Basic Building Blocks (Commits 1-3)

Started with individual modules (PC, IMEM, ALU, register file, control) and wired them into a single-cycle datapath. Fixed early bugs.

Phase 2: Complete Single-Cycle RV32I (Commits 4-6)

Added branches (BEQ, BNE, BLT, BGE, BLTU, BGEU), jumps (JAL, JALR), LUI and AUIPC. Complete single-cycle processor.

Phase 3: Pipelining (Commit 7)

The big rewrite. Split the datapath into 5 stages with pipeline registers. Every signal crossing a stage boundary needs to be registered.

Phase 4: Hazard Handling (Commits 8-9)

Added forwarding unit, load-use stall detection, branch flushing. Discovered and fixed the register file write-through bug.

Phase 5: Quantitative Analysis (Commits 10-11)

Added performance counters, Fibonacci benchmark, hazard test. Now we could measure CPI, stall rates, and flush rates.

Phase 6: Custom ISA Extension (Commit 12)

Added ROL instruction, ChaCha20 benchmark, Python hex generator. The capstone --showing the pipeline is correct, performant, and extensible.

Summary of Key Design Choices

Decision	Choice	Why
ISA	RISC-V RV32I	Open, clean, extensible
Architecture	Harvard	Avoids fetch/mem structural hazards
Pipeline depth	5 stages	Classic throughput vs complexity balance
Data hazards	Forwarding	Eliminates ~90%+ of stalls
Load-use	1-cycle stall	Data not available to forward yet
Control hazards	Flush	Simple, 1-cycle penalty
Branch resolve	EX stage	Reuses ALU for comparison
Memory width	32-bit words	Sufficient, avoids byte alignment
Custom instr	ROL	Real crypto need, minimal HW cost
Benchmark	ChaCha20 QR	Production cipher, rotation-heavy
Verification	Python golden model	Automated SW/HW co-verification

*This processor runs real programs, handles real hazards,
and demonstrates real ISA extensibility.
It is not a textbook diagram -- it is working hardware.*